

The complexity of cyclic shop scheduling problems

Nicholas G. Hall^{1,*†}, Tae-Eog Lee^{2,‡} and Marc E. Posner^{3,§}

¹*Department of Management Sciences, The Ohio State University, 2100 Neil Avenue,
Columbus, OH 43210-1144, U.S.A.*

²*Department of Industrial Engineering, Korea Advanced Institute of Science and Technology,
373-1, Guseong-dong, Yuseong-Gu, Taejeon 305-701, Korea*

³*Industrial Engineering, The Ohio State University, 1971 Neil Avenue, Columbus, OH 43210-1271, U.S.A.*

SUMMARY

We consider scheduling problems for shops in which a job set is manufactured repetitively. Jobs are scheduled to minimize the cycle time of the job set, which is equivalent to maximizing the throughput rate. We characterize the complexity of the scheduling problem for several types of job shops. Polynomial time algorithms are presented for open shops, and for job shops where each job has at most two operations. When there are two machines and the maximum number of operations of any job is a constant $k \geq 3$, the recognition version of the job shop problem is shown to be binary NP-complete. We describe a pseudopolynomial time algorithm for a special case of the problem when $k = 3$. We also establish that some generalizations of this problem are unary NP-complete. One consequence of these results is that the recognition version of the two machine job shop makespan problem with at most five operations per job is unary NP-complete. This resolves a question posed by Lenstra *et al.* (Ann. Discrete Math. 1977; 1:343). More generally, our results provide a map of the computational complexity of cycle time minimization problems that is analogous to that in the literature for makespan problems. Copyright © 2002 John Wiley & Sons, Ltd.

KEY WORDS: manufacturing; cyclic scheduling; sequencing; algorithms and complexity

1. INTRODUCTION

In *cyclic scheduling* problems, multiple types of jobs are processed and each job type has multiple copies. Jobs are produced repetitively in *job sets*. Typically, the jobs are produced

*Correspondence to: Nicholas G. Hall, Department of Management Sciences, The Ohio State University, 2100 Neil Avenue, Columbus, Ohio 43210-1144, U.S.A.

†E-mail: hall.33@osu.edu

‡E-mail: telee@mail.kaist.ac.kr

§E-mail: posner.1@osu.edu

Contract/grant sponsor: National Science Foundation; contract/grant number: DMI-9821033

Contract/grant sponsor: Max M. Fisher College of Business

Contract/grant sponsor: National Science Foundation; contract/grant number: DMI-9908437

in the same proportion as they appear in the production requirement [1]. For example, if the production requirement is 100 units of job A, 200 units of job B, and 300 units of job C, the job set is (1A, 2B, 3C). After the job set is determined and the operations for each job are assigned to the machines, the processing order in which the operations are processed at a machine must be specified. This processing order, called the *operation sequence*, is the same for each job set. In cyclic scheduling, the problem of finding the operation sequence that minimizes the cycle time is the *sequencing problem*.

Cyclic scheduling is effective when setup times and costs are small. Advantages over conventional scheduling models, such as batch scheduling or random dispatching rules, include better utilization of machines due to simultaneous production of complementary items [2], reduced scheduling complexity, simplified flow control due to predictable shop behaviour, continuous and smooth supply of complete part sets for downstream assembly, and timely delivery and reduced inventories [3, 4].

We briefly review the literature on cyclic scheduling problems. Hanen [5] discusses a generalized version of cyclic job shops in the context of task scheduling in pipelined computer architectures where there are precedence relations among the operations of different job sets. She shows that the sequencing problem for general job shops is unary NP-hard (also called strongly NP-hard [6]). However, she considers only the subset of feasible schedules where the start time of each job is later by the same fixed constant than the corresponding job in the previous job set. This constant is called the cycle time. Such a schedule is said to be *stable*. Carlier and Chrétienne [7] discuss the dominance of schedules that repeat periodically in a cyclic job shop. They show that a special case with only one type of resource constraint is efficiently solvable.

There are other studies of cyclic scheduling problems that consider constraints such as blocking [8], no-wait in process [9, 10], or reentrant job flows [11, 12] and that optimize different measures [13–15]. Kamoun and Sriskandarajah [16] review algorithmic and complexity issues for cyclic sequencing problems with blocking or no-wait constraints. McCormick and Rao [17] show that cyclic scheduling where the number of operations is limited only by the number of machines is an intractable problem. Zhang and Graves [18] discuss cyclic scheduling in an environment with stochastic machine failures, and provide a formulation as a convex program.

In this paper, we discuss the computational complexity of scheduling problems that minimize the cycle time in a variety of machine shops. We provide a polynomial time algorithm for open shop problems. We also describe a polynomial time algorithm for the job shop problem where each job has at most two operations. When jobs have a fixed number of operations k , where $k \geq 3$, the recognition version of the two machine job shop problem is shown to be binary NP-complete (also called ordinary NP-complete [6]). This result holds even for the special case where the ordering of the jobs by processing time is the same for all operations. For this case, we provide a pseudopolynomial time algorithm. When $k \geq 5$, the problem is shown to be unary NP-complete. Moreover, the analysis establishes that when $k \geq 5$, the recognition version of the two machine job shop problem with the makespan objective is unary NP-complete. This resolves an important open question discussed in Lenstra *et al.* [19]. A related cycle time minimization problem with three or more machines and $k \geq 3$ is also shown to be unary NP-complete.

2. NOTATION AND PROBLEM DESCRIPTION

Let $N = \{1, \dots, n\}$ denote the set of jobs to be processed. Each job $j \in N$ must complete n_j operations. For job shop problems, the sequence of operations is given for each job. We let (j, k) denote the k th operation of job j for $k \in \{1, \dots, n_j\}$. If two consecutive operations of a job are processed on the same machine, then they can be combined into one operation. Therefore, we assume without loss of generality that, for each job, operation $k + 1$ is on a different machine than operation k . The index set of machines is denoted by M , and each operation is assigned to a machine, M_i , $i \in M$. Operation (j, k) , where $j \in N$ and $k \in \{1, \dots, n_j\}$, is processed non-preemptively for $p_{j,k}$ units of time. Where it simplifies the exposition, we add the redundant first subscript i to denote that machine M_i is used for processing that operation. Thus, $p_{i,j,k}$ is the processing time for the k th operation of job j on machine M_i .

For each machine, the sequencing decision determines the order in which the operations are processed. Only after all the operations of a job set on a machine are completed can an operation of the next job set begin. However, a job of one job set can be processed on machine M_i while another job of the next job set is being processed on machine M_j , where $j \neq i$. We consider cyclic machine shops that have sufficient buffer capacity and are free from restrictions such as blocking or no-wait in process. Other assumptions include deterministic processing times, negligible setup and transportation times, and no machine breakdowns.

A given operation sequence may not specify a unique set of relative starting times for operations. Let $x_{j,k}^r$ be the starting time of operation (j, k) of the r th job set. A schedule is called *stable* if $x_{j,k}^{r+1} - x_{j,k}^r = \mu$ for some constant μ and for all $j \in N$, $k \in \{1, \dots, n_j\}$ and $r \geq 1$.

Lee and Posner [4] define a cycle time measure that is applicable to any schedule and any number of job sets produced. Let $\alpha(i)$ and $\beta(i)$ be the first and the last operation of a job set on machine M_i , respectively. Similarly, let $x_{\alpha(i)}^r$ denote the start time of operation $\alpha(i)$ in the r th job set, and $p_{\alpha(i)}$ the processing time of this operation. For a given schedule of r job sets, the *cycle time* is

$$\mu(r) = \max_{i \in M} \left\{ \frac{x_{\beta(i)}^r + p_{\beta(i)} - x_{\alpha(i)}^1}{r} \right\}$$

Observe that $x_{\beta(i)}^r + p_{\beta(i)} - x_{\alpha(i)}^1$ is the time it takes to process r job sets on machine M_i . The cycle time can be interpreted as the average time it takes to produce a job set. Lee and Posner [4] show that $\mu(r) = \mu(1)$ for $r \geq 1$ in a stable schedule. Notice that $\mu(1)$ can be interpreted as the schedule width of a single job set.

We observe that minimizing the cycle time is equivalent to maximizing the throughput rate.

Proposition 1

There exists a stable schedule that minimizes the cycle time. Further, the minimum cycle time is independent of the number of job sets and can be computed in $O(n^3)$ time.

Lee and Posner [4] establish this result for job shops. Consequently, this result also holds for flow shops. Moreover, it can be extended to open shops. Proposition 1 implies that we only need to determine an operation sequence for a single job set that minimizes $\mu(1)$. We follow the notation scheme of Graham *et al.* [20] for classifying scheduling problems. Kamoun and

Sriskandarajah [16] let C_t denote the cycle time. Thus, we let $C_t(\sigma)$ denote the cycle time of schedule σ . When the schedule is clear from context, we omit the argument.

The *workload* of a machine is the total time for which that machine is processing jobs, and is independent of scheduling decisions.

Proposition 2

The minimum cycle time is at least as large as the maximum workload on any machine.

The makespan ($C_{\max}$) and cycle time objectives are similar in that they both minimize the maximum completion time. There is, however, a major difference. The makespan objective minimizes the length of time from when production starts until it ends. Whereas, the cycle time objective minimizes the maximum elapsed time during which any machine is producing a single job set. Consequently, algorithmic or complexity results for a problem with a makespan objective are not necessarily valid for a problem with a cycle time objective, and *vice versa*.

3. A POLYNOMIAL TIME ALGORITHM FOR $J|n_j \leq 2|C_t$

We present an algorithm that finds an optimal schedule when each job has at most two operations.

Algorithm JOB2

1. Compute $t = \max_{i \in M} \{\sum_{j \in N} p_{i,j,1}\}$.
2. For each machine M_i , $i \in M$, schedule the operations of $\{(j, 1) | j \in N\}$ without idle time between them so that the last operation completes at t , and schedule the operations of $\{(j, 2) | j \in N\}$ without idle time between them so that the first operation starts at t . The processing orders within the sets $\{(j, 1) | j \in N\}$ and $\{(j, 2) | j \in N\}$ are arbitrary.
3. Output σ^* , the schedule constructed in Step 2. Output $C_t = \max_{i \in M} \{\sum_{k=1}^2 \sum_{j \in N} p_{i,j,k}\}$.

Figure 1 illustrates an example of a schedule generated by this algorithm. Theorem 1 establishes that JOB2 finds an optimal schedule.

Theorem 1

Algorithm JOB2 finds a minimum cycle time schedule for problem $J|n_j \leq 2|C_t$ in $O(n)$ time.

Proof

Consider the schedule of a single job set generated by JOB2. Since the operations of $\{(j, 1) | j \in N\}$ have no predecessor operations, they can be scheduled so that there is no idle time

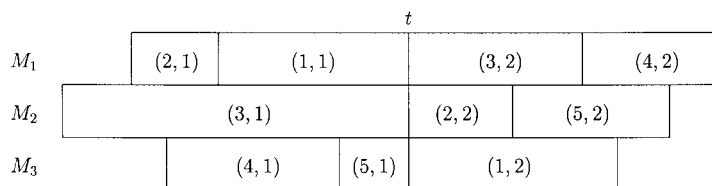


Figure 1. An example of a schedule generated by JOB2.

between them. Each operation in $\{(j, 2) | j \in N\}$ has all its predecessor operations completed by time t . Consequently, these operations also can be scheduled without idle time, starting at t . By repeating Algorithm JOB2, each subsequent job set can be scheduled with no idle time between any pair of operations on some machine. The workload of this machine is equal to the cycle time. Hence, the cycle time is equal to the maximum workload, which is $\max_{i \in M} \{\sum_{k=1}^2 \sum_{j \in N} p_{i,j,k}\}$. Since Proposition 2 establishes that no schedule can have a smaller cycle time, the schedule is optimal.

To establish the time complexity of Algorithm JOB2, we first note that $|M| \leq \sum_{j \in N} n_j = O(n)$. Step 1 requires $O(n)$ additions and $|M| - 1 = O(n)$ comparisons. Step 2 requires $O(n)$ time. Finally, Step 3 requires $O(n)$ additions and $|M| - 1 = O(n)$ comparisons. Thus, the overall time complexity of JOB 2 is $O(n)$. $\square$

Since an instance of problem $J|n_j \leq 2|C_t$ contains $O(n)$ processing times $p_{i,j,k}$, Algorithm JOB2 runs in polynomial time.

4. INTRACTABLE JOB SHOP PROBLEMS

Given a schedule, its feasibility and cycle time can be found in $O(\sum_{j=1}^n n_j)$ time. Consequently, the recognition versions of all problems considered in this section belong to the class NP. First, we establish that if each job has at most k operations, where $k \geq 3$ is a constant, then the recognition version of problem $J2|n_j \leq k|C_t$ is binary NP-complete. More general problems are shown to be unary NP-complete.

Theorem 2

The recognition version of problem $J2|n_j \leq k|C_t$ is binary NP-complete for any fixed $k \geq 3$.

Proof

Lev and Adiri [12] consider the reentrant flow shop problem where the objective is to minimize the makespan. When the jobs are processed first on machine M_1 , then on machine M_2 , and then on machine M_1 , they show that the problem is binary NP-complete. Since the time from the start of processing to its completion on machine M_1 is equal to the cycle time, this problem is a special case of $J2|n_j = 3|C_t$, where all jobs start on machine M_1 . Consequently, the recognition version of problem $J2|n_j \leq k|C_t$ is binary NP-complete for $k \geq 3$. $\square$

We now show that two generalizations of the recognition version of $J2|n_j \leq k|C_t$ for fixed $k \geq 3$ are unary NP-complete. Theorem 3 considers a generalization to $k \geq 5$. Theorem 4 considers a three machine job shop with $k \geq 3$. In our reductions, we use the following problem, which is known to be unary NP-complete.

3-Partition [6]

Given $3l$ elements with integer sizes $e_1, \dots, e_{3l}$, where $\sum_{i=1}^{3l} e_i = ly$ and $y/4 < e_i < y/2$ for $i = 1, \dots, 3l$, does there exist a partition $S_1, \dots, S_l$ of the index set $\{1, \dots, 3l\}$ such that $|S_j| = 3$ and $\sum_{i \in S_j} e_i = y$, for $j = 1, \dots, l$?

As part of this definition, we assume that if a solution to 3-Partition exists, then the elements are numbered such that $e_{3j-2} + e_{3j-1} + e_{3j} = y$ for $j = 1, \dots, l$.

M_1	$3l+1$	$4l+1$	$3l+1$	$3l+2$	$3l+1$	$3l+2$	$3l+3$	$\dots$	$4l$	$4l+2$	$4l$
M_2	$4l+1$	$3l+1$	$1, 2, 3$	$3l+1$	$3l+2$	$4, 5, 6$	$3l+2$	$\dots$	$3l-2,$ $3l-1,$ $3l$	$4l$	$4l+2$

Figure 2. A schedule with $C_t = (l+1)L + ly + l + 1$.**Theorem 3**

The recognition version of problem $J2|n_j \leq k|C_t$ is unary NP-complete for any fixed $k \geq 5$.

Proof

It suffices to prove the result for $k=5$. We use a reduction from 3-Partition that is polynomial in the size of the input. Consider the following instance of $J2|n_j \leq 5|C_t$:

$$\begin{aligned}
 n &= 4l+2, \\
 n_j &= 1, & j=1, \dots, 3l \\
 n_j &= 5, & j=3l+1, \dots, 4l \\
 n_j &= 2, & j=4l+1, 4l+2 \\
 p_{2,j,1} &= e_j, & j=1, \dots, 3l \\
 p_{1,j,1} &= 1, & j=3l+1, \dots, 4l \\
 p_{2,j,2} &= L, & j=3l+1, \dots, 4l \\
 p_{1,j,3} &= y, & j=3l+1, \dots, 4l \\
 p_{2,j,4} &= 1, & j=3l+1, \dots, 4l \\
 p_{1,j,5} &= L, & j=3l+1, \dots, 4l \\
 p_{2,4l+1,1} &= 1 \\
 p_{1,4l+1,2} &= L \\
 p_{1,4l+2,1} &= 1 \\
 p_{2,4l+2,2} &= L \\
 C &= (l+1)L + ly + l + 1
 \end{aligned}$$

where C denotes the threshold cycle time and $L=2/y$. We show that there exists a solution to 3-Partition if and only if there exists a schedule σ for this instance with $C_t(\sigma) \leq (l+1)L + ly + l + 1$.

($\Rightarrow$) Let the operations sequences for machines M_1 and M_2 be $((3l+1,1), (4l+1,2), (3l+1,3), (3l+2,1), (3l+1,5), (3l+2,3), (3l+3,1), \dots, (4l,3), (4l+2,1), (4l,5))$ and $((4l+1,1), (3l+1,2), (1,1), (2,1), (3,1), (3l+1,4), (3l+2,2), (4,1), (5,1), (6,1), (3l+2,4), \dots, (3l-2,1), (3l-1,1), (3l,1), (4l,4), (4l+2,2))$, respectively (see Figure 2). The first operation on each machine starts at time zero. Since a solution to 3-Partition exists, operations $(3s-2,1)$, $(3s-1,1)$ and $(3s,1)$ require exactly as much total processing time on machine M_2 as operation $(3l+s,3)$ does on machine M_1 , for $s=1, \dots, l$. Also, every other operation on one machine has a corresponding operation on the other machine with the same starting and completion time. Consequently, there is no idle time on either machine in the schedule. The cycle time therefore equals the workload on each machine, which is $(l+1)L + ly + l + 1$.

($\Leftarrow$) Since the workload on each machine is $(l+1)L + ly + l + 1 = C$, there is no idle time on either machine in a feasible schedule σ if $C_t(\sigma) \leq C$. Let $J = \{3l+1, \dots, 4l\}$. We assume without loss of generality that the identical jobs of J complete their second operations in index order.

The time between operations 1 and 3 of each job $j \in J$ is at least L , which is the time it takes to complete operation 2. Similarly, the time between operations 3 and 5 is at least 1. From the previous paragraph, there cannot be machine idle time in a feasible schedule that satisfies $C_i(\sigma) \leq C$. Consequently, machine M_1 must process other operations during the time between these operations. Thus, there are l intervals of time L on machine M_1 that must be filled. The only operations that can completely fill these intervals are operation 5 of the jobs in J and operation 2 of job $4l + 1$. Since, by assumption, $3l + 1$ is the first job in J to complete operation 2, it follows that operation 2 of job $4l + 1$ must be processed on machine M_1 between operations 1 and 3 of $3l + 1$; otherwise, there is idle time on machine M_1 . Similarly, operation 5 of job $3l + s$ must be processed on machine M_1 between operations 1 and 3 of job $3l + s + 1$, for $s = 1, \dots, l - 1$.

The only operations available to fill the l intervals of size 1 between operations 3 and 5 of the jobs in J are the first operations of jobs $3l + 2, \dots, 4l$, and $4l + 2$. Since the only operation that can fill this interval for job $4l$ is the first operation of job $4l + 2$, it follows that $4l + 2$ must be the last job to start processing on machine M_1 .

Thus, the only processing sequence on machine M_1 that does not have idle time is $((3l + 1, 1), (4l + 1, 2), (3l + 1, 3), (3l + 2, 1), (3l + 1, 5), (3l + 2, 3), (3l + 3, 1), \dots, (4l, 3), (4l + 2, 1), (4l, 5))$.

On machine M_2 , there is an interval of time y between operations 2 and 4 of each job $j \in J$. Observe that if more than y time units are processed between these operations, then operations 4 and 5 of job j are delayed. This causes idle time on machine M_1 . Consequently, operation 2 of job $4l + 2$ cannot be used to fill any of the intervals. Thus, the first operations of jobs $1, \dots, 3l$ are the only operations available to fill the intervals. Since the total processing time of these operations, ly , equals the total time in these intervals, there is no idle time if and only if each interval is exactly filled. This implies that none of jobs $1, \dots, 3l$ has its unique operation processed before operation $(4l + 1, 1)$ on machine M_2 . Hence, the processing of the jobs $1, \dots, 3l$ on machine M_2 provides a solution to 3-Partition. $\square$

Lenstra *et al.* [19] show that the recognition version of $J2|n_j \leq 3|C_{\max}$ is binary NP-complete. However, they leave open the important question of whether there exists a fixed value of k for which the recognition version of $J2|n_j \leq k|C_{\max}$ is unary NP-complete. We can show that the recognition version of this problem is unary NP-complete, by using a proof identical to that of Theorem 3 but with C representing a threshold makespan ($C_{\max}$) value.

Theorem 4

The recognition version of problem $J3|n_j \leq k|C_t$ is unary NP-complete for any fixed $k \geq 3$.

Proof

It suffices to prove the result for $k = 3$. We use a reduction from 3-Partition that is polynomial in the size of the input. Consider the following instance of $J3|n_j \leq 3|C_t$:

$$\begin{aligned} n &= 4l + 1 \\ p_{3,j,1} &= 1, & j &= 1, \dots, 3l \\ p_{2,j,2} &= 2ne_j, & j &= 1, \dots, 3l \\ p_{1,j,3} &= 1, & j &= 1, \dots, 3l \\ p_{1,3l+1,1} &= 1 \\ p_{2,3l+1,2} &= 1 \end{aligned}$$

M_1	$(3l+1, 1), \dots, (4l, 1)$					$(4l+1, 1)$		$(1, 3), \dots, (3l, 3)$	
M_2	$(3l+1, 2)$	$(1, 2)$	$(2, 2)$	$(3, 2)$	$(3l+2, 2)$	$\dots$	$(3l-2, 2)$	$(3l-1, 2)$	$(3l, 2)$
M_3	$(1, 1), \dots, (3l, 1)$			$(3l+1, 3), \dots, (4l, 3)$					$(4l+1, 3)$

Figure 3. A schedule with $C_l = 2lny + 4l + 1$.

$$\begin{aligned}
 p_{3,3l+1,3} &= 2ny + 1 \\
 p_{1,j,1} &= 2ny + 1, & j &= 3l+2, \dots, 4l \\
 p_{2,j,2} &= 1, & j &= 3l+2, \dots, 4l \\
 p_{3,j,3} &= 2ny + 1, & j &= 3l+2, \dots, 4l \\
 p_{1,4l+1,1} &= 2ny + 1 \\
 p_{2,4l+1,2} &= 1 \\
 p_{3,4l+1,3} &= 1 \\
 C &= 2lny + 4l + 1
 \end{aligned}$$

We show that there exists a solution to 3-Partition if and only if there exists a schedule σ for this instance with $C_l(\sigma) \leq C = 2lny + 4l + 1$.

($\Rightarrow$) Let the operations sequences for machines M_1 , M_2 and M_3 be $((3l+1, 1), \dots, (4l+1, 1), (1, 3), \dots, (3l, 3))$, $((3l+1, 2), (1, 2), (2, 2), (3, 2), (3l+2, 2), \dots, (3l-2, 2), (3l-1, 2), (3l, 2), (4l+1, 2))$, and $((1, 1), \dots, (3l, 1), (3l+1, 3), \dots, (4l+1, 3))$, respectively. Machines M_1 , M_2 and M_3 begin processing at times $3l-2$, $3l-1$ and 0 , respectively (see Figure 3).

Since a solution to 3-Partition exists, $p_{2,3s-2,2} + p_{2,3s-1,2} + p_{2,3s,2} = 2ny$ for $s = 1, \dots, l$. Thus, operation $(3l+s+1, 2)$ can begin as soon as operation $(3s, 2)$ completes. Further, operation $(3l+s+1, 3)$ can begin one time unit later. Consequently, this schedule has no idle time between any pair of operations in the job set. Because the workloads of machine M_1 and machine M_3 are both $2lny + 4l + 1$, and the workload of machine M_2 is $2lny + l + 1$, it follows that $C_l = 2lny + 4l + 1$.

($\Leftarrow$) Suppose that we have a schedule σ with $C_l(\sigma) = 2lny + 4l + 1$. Since the workload of machine M_1 and machine M_3 is $2lny + 4l + 1$, in any feasible schedule there cannot be idle time between the processing of any operations on machine M_1 or M_3 . Without loss of generality, we assume that machine M_3 starts processing at time 0 . Negative starting times on the other machines are permitted if necessary.

Because of the preceding operations on machines M_3 and M_2 , the jobs in $\{1, \dots, 3l\}$ can complete on machine M_1 no earlier than time $1 + 2lny + 1$. Since there can be no idle time between any pair of operations on machine M_1 , processing on this machine can begin no earlier than time $-4l + 1$.

Suppose that the first job processed in $\{3l+1, \dots, 4l+1\}$ by machine M_2 completes at time $t \geq 3l+1$. Then, the set $\{3l+1, \dots, 4l+1\}$ completes processing on machine M_3 no earlier than time $t + (2lny + l) + 1 \geq 2lny + 4l + 2$. Because machine M_3 begins processing at time 0 , there is idle time on machine M_3 , a contradiction. Hence, the first operation in $\{(3l+1, 2), \dots, (4l+1, 2)\}$ processed by machine M_2 completes no later than time $3l$. Since machine M_3 begins processing at time 0 and $\min_{j=1, \dots, 3l} \{p_{2,j,2}\} \geq 2n > 3l$, the first job processed by machine M_2 must be an element of $\{3l+1, \dots, 4l+1\}$. Otherwise, there is idle time on machine M_3 .

Because machine M_1 starts no earlier than time $-4l + 1$, and $p_{1,j,1} = 2ny + 1 > 7l - 1$ for $j = 3l + 2, \dots, 4l + 1$, the first job processed by machines M_1 and M_2 must be job $3l + 1$.

Let q be the last job in $\{3l + 2, \dots, 4l + 1\}$ processed by machine M_1 . Since machine M_1 starts processing no earlier than time $-4l + 1$, job q completes processing on machine M_1 no earlier than time $-4l + 1 + 1 + (2lny + l) = 2lny - 3l + 2$. If $q \neq 4l + 1$, then job q completes processing on machine M_3 no earlier than time $2lny - 3l + 2 + 1 + (2ny + 1) > 2lny + 4l + 1$. As a result, there is idle time on machine M_3 . This contradiction implies that $4l + 1$ is the last job processed from the set $\{3l + 2, \dots, 4l + 1\}$.

Since jobs $3l + 2, \dots, 4l$ are identical, we assume without loss of generality that these jobs are processed in index order. Since machine M_2 can start processing no earlier than time $-4l + 2$, the s th job of $3l + 1, \dots, 4l + 1$ processed by machine M_2 completes no earlier than time $2(s - 1)ny - 4l + s + 2$, for $s = 1, \dots, l + 1$.

Now, suppose that for some $s \in \{1, \dots, l\}$, job $3l + s \in \{3l + 1, \dots, 4l\}$ completes processing on machine M_2 at time $t \geq 2(s - 1)ny + 3l + s$. Then, job $3l + s$ completes processing on machine M_3 no earlier than time $t + 2ny + 1 \geq 2sny + 3l + s + 1$. Therefore, job $4l + 1$ completes on machine M_3 no earlier than time $2sny + 3l + s + 1 + (l - s)(2ny + 1) + 1 = 2lny + 4l + 2$. Hence, $C_t > 2lny + 4l + 1$, a contradiction. Consequently, job $3l + s$ completes processing on machine M_2 in the interval $[2(s - 1)ny - 4l + s + 2, 2(s - 1)ny + 3l + s - 1]$, for $s = 1, \dots, l$.

Consider the jobs in $\{1, \dots, 3l\}$ that are processed between jobs $3l + s$ and $3l + s + 1$ on machine M_2 for some $1 \leq s \leq l$. Suppose that the total processing time of these jobs is at least $2n(y + 1)$. Then, even if operation $(3l + s, 2)$ completes as early as possible, at time $2(s - 1)ny - 4l + s + 2$, job $3l + s + 1$ cannot complete until time

$$\begin{aligned} 2(s - 1)ny - 4l + s + 2 + 2n(y + 1) + 1 &= 2sny + 2n - 4l + s + 3 \\ &> 2sny + 3l + s \end{aligned}$$

This contradicts the completion time interval for operation $(3l + s + 1, 2)$.

Alternatively, suppose that the total processing time of the jobs in $\{1, \dots, 3l\}$ that are processed on machine M_2 between jobs $3l + s$ and $3l + s + 1$ is at most $2n(y - 1)$. Now, all the operations $\{(1, 2), \dots, (3l, 2)\}$ are processed between operations $(3l + 1, 2)$ and $(4l + 1, 2)$. Then, there exists an i , $1 \leq i \neq s \leq l$, such that the total processing time between operations $(3l + 1, 2)$ and $(3l + i + 1, 2)$ is at least $2n(y + 1)$. This again implies a contradiction.

Hence, the processing order of the jobs in $\{1, \dots, 3l\}$ on machine M_2 provides a solution to 3-Partition. $\square$

5. A SPECIAL CASE OF PROBLEM $J2|n_j \leq 3|C_t$

Let $N_i = \{j \in N \mid \text{job } j \text{ has exactly } i \text{ operations}\}$ for $i = 1, 2, 3$. Without loss of generality, we assume throughout this section that $\sum_{j \in N_3} p_{1,j,2} \geq \sum_{j \in N_3} p_{2,j,2}$. Let $N' \subseteq N_3$ be the set of jobs for which the first operation is on machine M_2 , and let $n' = |N'|$. Reindex the jobs so that $N' = \{1, \dots, n'\}$. We consider a special case of problem $J2|n_j \leq 3|C_t$ under the condition that

$$p_{2,j,1} \leq p_{2,s,1} \text{ iff } p_{1,j,2} \leq p_{1,s,2} \text{ iff } p_{2,j,3} \leq p_{2,s,3}, \quad \text{for all } j, s \in N' \quad (1)$$

Condition (1) implies that if one of the operations of a job $j \in N'$ has a smaller processing time than the same operation of another job $s \in N'$, then all three operations do. This condition occurs, for example, in manufacturing environments where the processing time on each machine depends on the physical size of a job. In this case, a large part needs more processing time at each operation than a small part. For example, a large part may have more surface area, which requires more time for the operations of grinding, polishing and finishing.

Theorem 2 demonstrates that there does not exist a polynomial time algorithm for the general problem $J2|n_j \leq 3|C_t$, unless $P = NP$. We now demonstrate the stronger result that a polynomial time algorithm does not exist for the special case of the problem under Condition (1), unless $P = NP$. We use a reduction from the following problem, which is known to be binary NP-complete.

Partition [6]

Given l elements with integer sizes $e_1, \dots, e_l$, does there exist a partition S_1, S_2 of the index set $\{1, \dots, l\}$ such that $\sum_{i \in S_1} e_i = \sum_{i \in S_2} e_i$?

As part of this definition, we assume that if a solution to Partition exists, then the elements are numbered such that $S_1 = \{1, \dots, s\}$ and $S_2 = \{s + 1, \dots, l\}$.

Theorem 5

The recognition version of problem $J2|n_j \leq k|C_t$ under Condition (1) is binary NP-complete for any fixed $k \geq 3$.

Proof

It suffices to prove the result for $k = 3$. We use a reduction from Partition that is polynomial in the size of the input. Consider the following instance of $J2|n_j \leq 3|C_t$:

$$\begin{aligned} n &= l + 2 \\ p_{2,j,1} &= 1, & j &= 1, \dots, l \\ p_{1,j,2} &= 2ne_j, & j &= 1, \dots, l \\ p_{2,j,3} &= 1, & j &= 1, \dots, l \\ p_{2,l+1,1} &= n \sum_{j=1}^l e_j - l + 1 \\ p_{1,l+1,2} &= 2n \sum_{j=1}^l e_j \\ p_{2,l+1,3} &= n \sum_{j=1}^l e_j - l + 1 \\ p_{1,l+2,1} &= 1 \\ p_{2,l+2,2} &= 2n \sum_{j=1}^l e_j \\ p_{1,l+2,3} &= 1 \\ C &= 4n \sum_{j=1}^l e_j + 2 \end{aligned}$$

where C denotes the threshold cycle time. We show that there is a solution to Partition if and only if there exists a schedule for this instance with $C_t \leq 4n \sum_{j=1}^l e_j + 2$.

M_1	$(l+2, 1)$	$(1, 2), \dots, (s, 2)$	$(l+1, 2)$	$(s+1, 2), \dots, (l, 2)$	$(l+2, 3)$
M_2	$(1, 1), \dots, (l, 1)$	$(l+1, 1)$	$(l+2, 2)$	$(l+1, 3)$	$(1, 3), \dots, (l, 3)$

Figure 4. A schedule with $C_t = 4n \sum_{j=1}^l e_j + 2$.

($\Rightarrow$) Let the operation sequences for machines M_1 and M_2 be $((l+2, 1), (1, 2), \dots, (s, 2), (l+1, 2), (s+1, 2), \dots, (l, 2), (l+2, 3))$ and $((1, 1), \dots, (l, 1), (l+1, 1), (l+2, 2), (l+1, 3), (1, 3), \dots, (l, 3))$, respectively (see Figure 4). The first operation on each machine starts at time 0. Since a solution to Partition exists, $\sum_{j=1}^s p_{1,j,2} = n \sum_{j=1}^l e_j$, and operation $(l+1, 2)$ can begin processing at time $n \sum_{j=1}^l e_j + 1$. Further, operation $(l+1, 3)$ can begin $2n \sum_{j=1}^l e_j$ time units later. Consequently, this schedule can complete without idle time. The cycle time equals the workload of machine M_1 and machine M_2 , which is $4n \sum_{j=1}^l e_j + 2$.

($\Leftarrow$) Since the workload for both machines 1 and 2 is $4n \sum_{j=1}^l e_j + 2 = C_t$, there is no idle time between any operations in a feasible schedule with $C_t \leq C$. Let $S_1 \subseteq \{1, \dots, l\}$ where $i \in S_1$ if operation $(i, 2)$ is processed on machine M_1 before operation $(l+1, 2)$, and let $S_2 = \{1, \dots, n\} \setminus S_1$. Without loss of generality, let machine M_2 start processing at time 0. A negative starting time on machine M_1 is permitted if necessary.

Suppose that $\sum_{j \in S_1} p_{1,j,2} > n \sum_{j=1}^l e_j$. This implies that $\sum_{j \in S_1} p_{1,j,2} \geq n \sum_{j=1}^l e_j + 2n$. Since the set of operations $\{(j, 2) | j \in S_1\}$ can start on machine M_1 no earlier than time 1, it can complete no earlier than time $n \sum_{j=1}^l e_j + 2n + 1$. Consequently, $(l+1, 2)$ completes no earlier than time $3n \sum_{j=1}^l e_j + 2n + 1$. This implies that $(l+1, 3)$ completes no earlier than time $3n \sum_{j=1}^l e_j + 2n + 1 + n \sum_{j=1}^l e_j - l + 1 > C_t$, a contradiction.

Alternatively, suppose that $\sum_{j \in S_1} p_{1,j,2} < n \sum_{j=1}^l e_j$. This implies that $\sum_{j \in S_2} p_{1,j,2} \geq n \sum_{j=1}^l e_j + 2n$. Consequently, machine M_2 finishes processing no earlier than the earliest completion time of $(l+1, 2)$ plus the processing times of $(j, 2)$ for all $j \in S_2$ and the processing time of $(i, 3)$ where $i \in S_2$ is the last operation processed by machine M_2 from S_2 . This time is $(3n \sum_{j=1}^l e_j - l + 1) + (n \sum_{j=1}^l e_j + 2n) + 1 > C_t$, a contradiction.

Thus, $\sum_{j \in S_1} p_{1,j,2} = n \sum_{j=1}^l e_j$, and (S_1, S_2) provides a solution to Partition. $\square$

In order to describe the complexity of problem $J2 | n_j \leq 3 | C_t$ under Condition (1) completely, we now develop an algorithm that runs in pseudopolynomial time. We define a *critical operation set* of a schedule to be a set of operations whose processing times determine the cycle time of the schedule. A small increase in the processing time of any operation in this set increases the cycle time by an equivalent amount.

Example 1

Consider an example with three machines and three jobs, where the processing times are: $p_{1,1,1} = 2$, $p_{2,1,2} = 3$, $p_{3,1,3} = 3$; $p_{1,2,1} = 1$, $p_{2,2,2} = 1$; $p_{3,3,1} = 1$, $p_{1,3,2} = 1$. A stable schedule for this example is presented in Figure 5, where the bold line separates one job set from the next. Operations $(1, 1)$, $(1, 2)$, $(1, 3)$, $(3, 1)$, and $(3, 2)$ form the critical operation set (see Lee and Posner [4], for details about how to find this set). The cycle time for this set is 5.

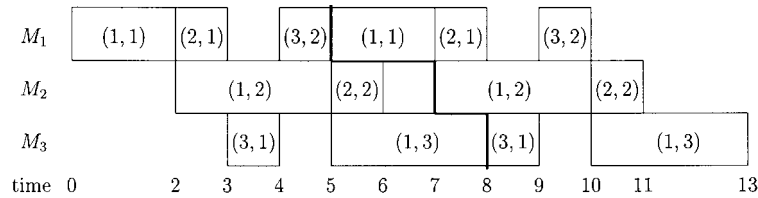


Figure 5. A stable schedule with a cycle time of 5.

An algorithm that runs in pseudopolynomial time is described. We begin with two lemmas that hold for the general problem $J2|n_j = 3|C_t$.

Lemma 1

In problem $J2|n_j = 3|C_t$, there exists an optimal schedule with the following characteristics.

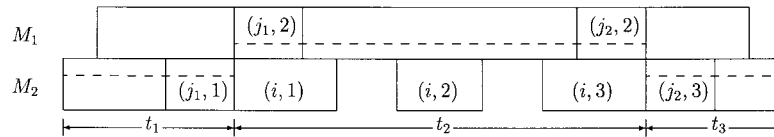
- On each machine M_1 and M_2 , all the first operations of jobs precede all the second operations of jobs, which precede all the third operations of jobs.
- All of the second operations on machine M_2 are processed concurrently with the second operations on machine M_1 .
- There is no idle time between any operations on machine M_1 . Also, there is no idle time between any pair of first operations, second operations or third operations on machine M_2 .

Proof

Let σ^* denote an optimal schedule. We construct a new schedule σ that satisfies Conditions (a), (b) and (c) of the lemma. On each machine, interchange the operations so that all of the first operations precede all of the second operations and all of the second operations precede all of the third operations. The sequences within the first operations, within the second operations, and within the third operations remain the same as in σ^* . Since $\sum_{j \in N} p_{1,j,2} \geq \sum_{j \in N} p_{2,j,2}$, we process all second operations on machine M_2 concurrently with the second operations on machine M_1 . Thus, machine M_2 never waits for a second operation, and machine M_1 never waits for a third operation. Therefore, we can schedule the first operations, followed by the second operations, followed by the third operations, on machine M_1 , without idle time between any pair of operations. Also, idle time between any pair of first operations on machine M_2 can be removed without increasing the cycle time, by starting operations earlier until all the idle time is between the first and second operations. Similarly, starting the third operations later on machine M_2 removes any idle time between them. An example of σ is presented in Figure 7 following this proof.

By construction, σ satisfies Conditions (a), (b) and (c) of the lemma. Thus, it only remains to prove that σ is an optimal schedule. First, suppose that $C_t(\sigma) = x_{\beta(1)}^1 + p_{\beta(1)} - x_{\alpha(1)}^1$. Because there is no idle time between any pair of operations on machine M_1 , σ has the same cycle time as σ^* . Hence, σ is an optimal schedule.

Alternatively, suppose that $C_t(\sigma) > x_{\beta(1)}^1 + p_{\beta(1)} - x_{\alpha(1)}^1$. When there is no idle time on M_2 , then σ is an optimal schedule. Consequently, we assume that there is idle time on machine M_2 that cannot be reduced by changing the starting times of the operations (without altering the operation sequence).

Figure 6. A critical operation set for σ in Lemma 1.

Therefore, any critical operation set in σ must contain the first and last operations on machine M_2 and also some operations on machine M_1 . Consider an arbitrary critical operation set of this type in σ . Let j_1 denote the job whose second operation is processed *first* from the critical operation set. Also, let j_2 denote the job whose second operation is processed *last* from the critical operation set. The critical operation set contains the set of first operations of jobs on machine M_2 that are sequenced from the start of processing through operation $(j_1, 1)$. Also, the critical operation set contains the operations sequenced on machine M_1 from $(j_1, 2)$ through operation $(j_2, 2)$. Finally, the critical operation set contains the operations sequenced on machine M_2 from $(j_2, 3)$ through the last operation. This set of operations is illustrated in Figure 6.

Let t_1^* denote the time from the start of the first operation on machine M_2 to the completion of operation $(j_1, 1)$ in schedule σ^* . Let t_2^* denote the time from the start of operation $(j_1, 2)$ to the completion of operation $(j_2, 2)$ on machine M_1 in schedule σ^* . Also, let t_3^* denote the time from the start of operation $(j_2, 3)$ to the completion of the last operation on machine M_2 in schedule σ^* . Let t_i have an equivalent meaning with respect to σ as t_i^* does with respect to σ^* , for $i = 1, 2, 3$. Note that operations $(j_1, 1)$, $(j_1, 2)$, $(j_2, 2)$ and $(j_2, 3)$ may not be in the critical operation set in σ^* . Therefore, since σ^* is a feasible schedule, operation $(j_1, 2)$ may start later on machine M_1 than operation $(j_1, 1)$ completes on machine M_2 , and operation $(j_2, 3)$ may start later on machine M_2 than operation $(j_2, 2)$ completes on machine M_1 . Thus, $C_t(\sigma^*) \geq t_1^* + t_2^* + t_3^*$.

By definition of the critical operation set, $C_t(\sigma) = t_1 + t_2 + t_3$. Now, since the operations preceding $(j_1, 1)$ on machine M_2 in schedule σ are a subset of those in σ^* , we have $t_1 \leq t_1^*$. Similarly, because the operations between $(j_1, 2)$ and $(j_2, 2)$ on machine M_1 in σ are a subset of those in σ^* , $t_2 \leq t_2^*$. Finally, since the operations following $(j_2, 3)$ on machine M_2 in σ are a subset of those in σ^* , $t_3 \leq t_3^*$. Consequently, $C_t(\sigma) = t_1 + t_2 + t_3 \leq t_1^* + t_2^* + t_3^* \leq C_t(\sigma^*)$. Thus, σ is an optimal schedule. $\square$

An example of the job rearrangement in Lemma 1 appears in Figure 7. The original schedule is shown in Figure 7(a). The modified schedule with the same cycle time is shown in Figure 7(b). In the modified schedule, on each machine, all first operations precede all second operations, which in turn precede all third operations. Also, there is no idle time between any pair of first operations, second operations or third operations on machine M_2 . Observe that the critical operation set in Figure 7(b) is $\{(3, 1), (2, 1), (2, 2), (2, 3)\}$.

A *block* of operations is a set of operations that are performed consecutively on one machine, without idle time between any of the operations. Let σ_k^i denote the k th operations of jobs that are processed on machine M_i , for $i = 1, 2$, $k = 1, 2, 3$. From Lemma 1, we only consider schedules where the jobs in N_3 on machine M_1 are in blocks $\sigma_1^1, \sigma_2^1, \sigma_3^1$ and the jobs in N_3 on machine M_2 are in blocks $\sigma_1^2, \sigma_2^2, \sigma_3^2$. The next lemma establishes the optimality of a permutation schedule for the jobs within these blocks.

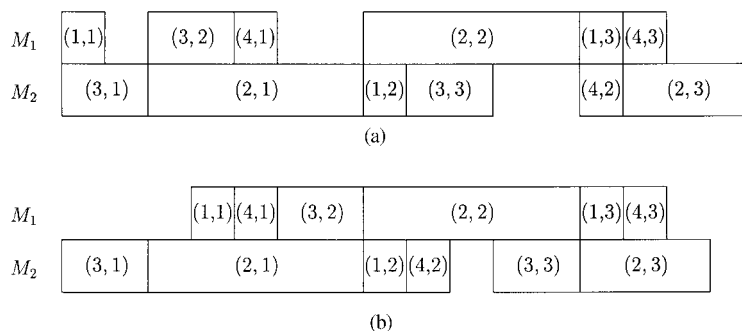


Figure 7. Example of the job rearrangement in Lemma 1.

Lemma 2

For problem $J2|n_j=3|C_t$, there exists an optimal schedule σ in which the sequences of operations in σ_1^1 , σ_2^2 and σ_3^3 are identical, and the sequences of operations in σ_1^2 , σ_2^1 and σ_3^2 are identical.

Proof

Consider an optimal schedule in which the sequence of operations $(j,1),(s,1)$ appears in block σ_1^1 , and the sequence $(s,2),(j,2)$ appears in σ_2^2 .

Suppose that operation $(j,3)$ precedes operation $(s,3)$ in σ_3^1 . Interchange operations $(s,2)$ and $(j,2)$. Since job j arrives at machine M_2 before job s and is processed again on machine M_1 at an earlier time than job s , no idle time is created. Thus, C_t is not increased, and the schedule remains optimal.

Alternatively, suppose that operation $(s,3)$ precedes operation $(j,3)$ in σ_3^1 . Interchange operations $(j,1)$ and $(s,1)$. Since job s is processed before job j on machine M_2 , no idle time is created. Thus, C_t is not increased, and the schedule remains optimal.

Symmetric arguments can be applied to subsequences σ_1^2 , σ_2^1 and σ_3^2 . $\square$

From Lemmas 1 and 2, we generate a schedule for the jobs in N_3 where on each machine, the first operations, the second operations and the third operations each form a block. Also, there is no idle time between any operations on machine M_1 , and block σ_2^2 starts no earlier than block σ_2^1 . The sequences within the blocks σ_1^2 , σ_2^1 and σ_3^2 and within the blocks σ_1^1 , σ_2^2 and σ_3^1 are identical. Since Lemma 1 establishes that any sequence for blocks σ_1^1 , σ_2^2 and σ_3^1 is optimal, we consider the sequence only for blocks σ_1^2 , σ_2^1 and σ_3^2 , which together make up the jobs in N' . To simplify the exposition, for the remainder of this section we shorten $p_{2,j,1}$ to $p_{j,1}$, $p_{1,j,2}$ to $p_{j,2}$ and $p_{2,j,3}$ to $p_{j,3}$ for $j \in N'$.

Wang *et al.* [15] study the reentrant flow shop problem, where all jobs start and end on the same machine. Minimizing makespan in the reentrant flow shop problem is a special case of problem $J2|n_j \leq 3|C_t$. We extend their approach by selecting a job $d \in N'$ to be a *dividing job*. Define d to be the job such that $(d,2)$ is the last operation that starts no later on machine M_1 than block σ_1^2 completes on machine M_2 . Let $A = \{j \in N' \mid (j,2) \text{ is processed on machine } M_1 \text{ before } (d,2)\}$ and $B = \{j \in N' \mid (j,2) \text{ is processed on machine } M_1 \text{ after } (d,2)\}$.

Observe that idle time on machine M_2 does not occur due to interactions between the jobs in A and those in B . Idle time is created only by interactions between the first and second

operations of the jobs in A or between the second and third operations of the jobs in B . We observe that a minimum idle time schedule for just the jobs in A can be found by sequencing the first two operations by Johnson's Rule [21]. A similar observation holds for the second and third operations of the jobs in B . Thus, for the general version of problem $J2|n_j=3|C_t$, it is straightforward to find an optimal sequence for each set once A and B are determined. Also, the jobs in $N_1 \cup N_2$ can be scheduled separately after those in N_3 . These observations motivate our algorithm.

If the order in which Johnson's Rule sequences the A and B jobs is the same, then we have a *consistent* job order for generating partial schedules. After a description of the algorithm, we show that (1) implies the existence of an optimal consistent job order. As a result, the jobs can be ordered before being partitioned into A and B . While the optimal composition of A and B is still difficult to determine, the optimal sequences for the jobs in A and B are automatically specified by the consistent job order.

We now describe a dynamic programming algorithm, JOB3, that provides in pseudopolynomial time an optimal schedule for problem $J2|n_j \leq 3|C_t$ under Condition (1). Our overall strategy is first to index the jobs with three operations in an order compatible with the way Johnson's Rule selects the items, i.e. by non-decreasing processing time of the shortest operation. Next, we select a job to be the dividing job. Each job in turn is considered as a possible dividing job. Then, we consider partial schedules where the next job is placed either in A or in B . By selecting the schedules for each possible state with the least idle time on machine M_2 , we create a pseudopolynomial time procedure.

When we build partial schedules, ten distinct blocks of operations are created. The total processing time for each of these blocks is an explicit or implicit state variable. For a specified partial schedule, let

$$\begin{aligned} A_{11} &= \sum_{j \in \{s \in A | p_{s,1} < p_{s,2}\}} p_{j,1} \\ A_{12} &= \sum_{j \in \{s \in A | p_{s,1} \geq p_{s,2}\}} p_{j,1} \\ A_{21} &= \sum_{j \in \{s \in A | p_{s,1} < p_{s,2}\}} p_{j,2} \\ A_{22} &= \sum_{j \in \{s \in A | p_{s,1} \geq p_{s,2}\}} p_{j,2} \\ A_3 &= \sum_{j \in A} p_{j,3} \\ B_1 &= \sum_{j \in B} p_{j,1} \\ B_{21} &= \sum_{j \in \{s \in B | p_{s,2} \leq p_{s,3}\}} p_{j,2} \\ B_{22} &= \sum_{j \in \{s \in B | p_{s,2} > p_{s,3}\}} p_{j,2} \\ B_{31} &= \sum_{j \in \{s \in B | p_{s,2} \leq p_{s,3}\}} p_{j,3} \\ B_{32} &= \sum_{j \in \{s \in B | p_{s,2} > p_{s,3}\}} p_{j,3} \end{aligned}$$

For a given d and a specified partial schedule, let

$$\begin{aligned} D_{A1} &= \text{elapsed time from the completion of } A_{11} \text{ to the completion of } A_{21} \\ D_{A2} &= \text{elapsed time from the completion of } A_{12} \text{ to the completion of } A_{22} \\ D_{B1} &= \text{elapsed time from the start of } B_{21} \text{ to the start of } B_{31} \\ D_{B2} &= \text{elapsed time from the start of } B_{22} \text{ to the start of } B_{32} \end{aligned}$$

The arrangement of these blocks and the associated elapsed times are shown in Figure 8. Note that there is no idle time within any of the blocks. In describing the algorithm, we use the name of a block and its total processing time interchangeably.

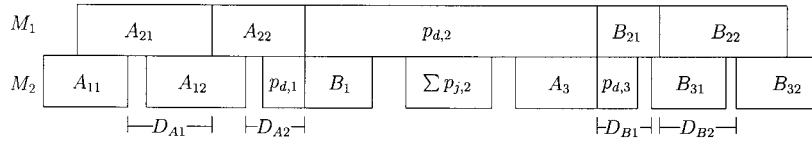


Figure 8. Blocks and elapsed times in a schedule produced by JOB3.

A state of the dynamic program is defined by $(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32})$, where d is the dividing job and each of the partial schedules $A_{11}, \dots, B_{32}$ contains the jobs $1, \dots, j$ for some $j \in N'$. Because A_3 , B_1 , and B_{22} are uniquely determined by jobs d and j and the other state variables, they are not included in the description of the state space. Specifically, $A_3 = \sum_{k=1, k \neq d}^j p_{k,3} - B_{31} - B_{32}$, $B_1 = \sum_{k=1, k \neq d}^j p_{k,1} - A_{11} - A_{12}$, and $B_{22} = \sum_{k=1, k \neq d}^j p_{k,2} - A_{21} - A_{22} - B_{21}$. Let $F(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32})$, the value function, be a set of vectors of the form $(D_{A1}, D_{A2}, D_{B1}, D_{B2})$.

After constructing a sequence for the jobs in N_3 , we schedule the jobs in $N_1 \cup N_2$ so that the idle time on machine M_2 decreases by as much as possible. For vectors $x, y \in \mathbb{R}^n$, we say that x dominates y if $x_i \leq y_i$ for $i = 1, \dots, n$.

Algorithm JOB3

0. Let $N_i = \{j \in N \mid j \text{ has } i \text{ operations}\}$ for $i = 1, 2, 3$.
 Index the machines so that $\sum_{j \in N_3} p_{1,j,2} \geq \sum_{j \in N_3} p_{2,j,2}$.
 Let $N' \subseteq N_3$ be the set of jobs for which the first operation is on machine M_2 , and let $n' = |N'|$.
 Index the jobs $j \in N_3$ so that $j_1 < j_2$ if $j_1 \in N'$ and $j_2 \in N_3 \setminus N'$.
 Index the jobs $1, \dots, n'$ such that $p_{1,2} \leq \dots \leq p_{n',2}$.
 Define $S = \{j \in N' \mid p_{j,2} \leq p_{j,1}, p_{j,3}\}$, $T = \{j \in N' \mid p_{j,2} > p_{j,1}, p_{j,3}\}$, $V = \{j \in N' \mid p_{j,1} \leq p_{j,2} \leq p_{j,3}\}$ and $W = \{j \in N' \mid p_{j,1} \geq p_{j,2} \geq p_{j,3}\}$.
 Set $F(d, 0, \dots, 0) = (0, 0, 0, 0)$ for $d = 2, \dots, n'$ and set $F(\cdot, \dots, \cdot) = \emptyset$ otherwise.
 Set $F(1, 1, 0, \dots, 0) = (0, 0, 0, 0)$.
 Set $d = 1$ and $j = 2$.
1. If $j \in S$, then go to Step 1a. If $j \in T$, then go to Step 1b. If $j \in V$, then go to Step 1c. If $j \in W$, then go to Step 1d.
 - a. For $j \in A$, set

$$F^A = \{(D_{A1}, \max\{D_{A2}, A_{22} - A_{12} + p_{j,2}\}, D_{B1}, D_{B2}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11}, A_{12} - p_{j,1}, A_{21}, A_{22} - p_{j,2}, B_{21}, B_{31}, B_{32})\}.$$

For $j \in B$, set

$$F^B = \{(D_{A1}, D_{A2}, \max\{D_{B1}, B_{21} - B_{31} + p_{j,2}\}, D_{B2}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11}, A_{12}, A_{21}, A_{22}, B_{21} - p_{j,2}, B_{31} - p_{j,3}, B_{32})\}.$$

Set $F(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32}) = F^A \cup F^B$. Go to Step 2.

[If $j \in A$, then operation $(j, 1)$ is scheduled at the start of block A_{12} , $(j, 2)$ is scheduled at the start of block A_{22} , and $(j, 3)$ is scheduled using Lemma 2 in block A_3 .

- If $j \in B$, then operation $(j, 1)$ is scheduled using Lemma 2 in block B_1 , $(j, 2)$ is scheduled at the end of block B_{21} , and $(j, 3)$ is scheduled at the end of block B_{31} .]
- b. For $j \in A$, set

$$F^A = \{(\max\{D_{A1} + p_{j,2} - p_{j,1}, p_{j,2}\}, D_{A2}, D_{B1}, D_{B2}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11} - p_{j,1}, A_{12}, A_{21} - p_{j,2}, A_{22}, B_{21}, B_{31}, B_{32})\}.$$

For $j \in B$, set

$$F^B = \{(D_{A1}, D_{A2}, D_{B1}, \max\{D_{B2} + p_{j,2} - p_{j,3}, p_{j,2}\}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32} - p_{j,3})\}.$$

Set $F(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32}) = F^A \cup F^B$. Go to Step 2.

[If $j \in A$, then operation $(j, 1)$ is scheduled at the end of block A_{11} , $(j, 2)$ is scheduled at the end of block A_{21} , and $(j, 3)$ is scheduled using Lemma 2 in block A_3 . If $j \in B$, then operation $(j, 1)$ is scheduled using Lemma 2 in block B_1 , $(j, 2)$ is scheduled at the start of block B_{22} , and $(j, 3)$ is scheduled at the start of block B_{32} .]

- c. For $j \in A$, set

$$F^A = \{(\max\{D_{A1} + p_{j,2} - p_{j,1}, p_{j,2}\}, D_{A2}, D_{B1}, D_{B2}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11} - p_{j,1}, A_{12}, A_{21} - p_{j,2}, A_{22}, B_{21}, B_{31}, B_{32})\}.$$

For $j \in B$, set

$$F^B = \{(D_{A1}, D_{A2}, \max\{D_{B1}, B_{21} - B_{31} + p_{j,2}\}, D_{B2}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11}, A_{12}, A_{21}, A_{22}, B_{21} - p_{j,2}, B_{31} - p_{j,3}, B_{32})\}.$$

Set $F(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32}) = F^A \cup F^B$. Go to Step 2.

[If $j \in A$, then operation $(j, 1)$ is scheduled at the end of block A_{11} , $(j, 2)$ is scheduled at the end of block A_{21} , and $(j, 3)$ is scheduled using Lemma 2 in block A_3 . If $j \in B$, then operation $(j, 1)$ is scheduled using Lemma 2 in block B_1 , $(j, 2)$ is scheduled at the end of block B_{21} , and $(j, 3)$ is scheduled at the end of block B_{31} .]

- d. For $j \in A$, set

$$F^A = \{(D_{A1}, \max\{D_{A2}, A_{22} - A_{12} + p_{j,2}\}, D_{B1}, D_{B2}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11}, A_{12} - p_{j,1}, A_{21}, A_{22} - p_{j,2}, B_{21}, B_{31}, B_{32})\}.$$

For $j \in B$, set

$$F^B = \{(D_{A1}, D_{A2}, D_{B1}, \max\{D_{B2} + p_{j,2} - p_{j,3}, p_{j,2}\}) \mid (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, j-1, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32} - p_{j,3})\}.$$

Set $F(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32}) = F^A \cup F^B$.

[If $j \in A$, then operation $(j, 1)$ is scheduled at the start of block A_{12} , $(j, 2)$ is scheduled at the start of block A_{22} , and $(j, 3)$ is scheduled using Lemma 2 in block A_3 . If $j \in B$, then operation $(j, 1)$ is scheduled using Lemma 2 in block B_1 ,

$(j, 2)$ is scheduled at the start of block B_{22} , and $(j, 3)$ is scheduled at the start of block B_{32} .]

2. Eliminate all dominated schedules from $F(d, j, A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32})$.
Set $j = j+1$ if $j+1 \neq d$. Otherwise, set $j = d+1$ and $F(d, d, \cdot, \dots, \cdot) = F(d, d-1, \cdot, \dots, \cdot)$.
If $j \leq n'$, then go to Step 1.
3. If $d < n'$, then set $d = d+1$, $j = 1$, and go to Step 1.
4. Sequence the first operations of $j \in N_1 \cup N_2$ that are on machine M_1 in arbitrary order at the start of block A_{21} , and set $A_{21} = A_{21} + \sum_{j \in N_1 \cup N_2} p_{1,j,1}$. Sequence the second operations of this set in arbitrary order at the end of block A_3 , and set $A_3 = A_3 + \sum_{j \in N_1 \cup N_2} p_{2,j,2}$. Sequence the first operations of $j \in N_1 \cup N_2$ that are on machine M_2 in arbitrary order at the start of block B_1 , and set $B_1 = B_1 + \sum_{j \in N_1 \cup N_2} p_{2,j,1}$. Sequence the second operations of this set in arbitrary order at the end of block B_{22} , and set $B_{22} = B_{22} + \sum_{j \in N_1 \cup N_2} p_{1,j,2}$.
5. For each $y = (D_{A1}, D_{A2}, D_{B1}, D_{B2}) \in F(d, n', A_{11}, A_{12}, A_{21}, A_{22}, B_{21}, B_{31}, B_{32})$, let

$$I^A = \max\{0, D_{A2} - p_{d,1}, D_{A1} + A_{22} - A_{12} - p_{d,1}\} - B_1$$

$$I^B = \max\{0, D_{B1} - p_{d,3}, D_{B2} + B_{21} - B_{31} - p_{d,3}\} - A_3, \text{ and}$$

$$I_y = \max\{0, I^A\} + \max\left\{0, p_{d,2} + \min\{0, I^A\} + \min\{0, I^B\} - \sum_{j \in N} p_{2,j,2}\right\} + \max\{0, I^B\}$$

[I^A denotes the idle time on machine M_2 before the start of operation $(d, 2)$, and I^B denotes the idle time on machine M_2 after the end of operation $(d, 2)$. Also, I_y denotes the total idle time in the schedule associated with vector y .]

6. Let I_{y^*} denote the minimum idle time over all schedules considered in Step 5.

Output σ^* , the schedule corresponding to I_{y^*} .

Output $C_t = \max\{\sum_{j \in N} (p_{1,j,1} + p_{1,j,2} + p_{1,j,3}), \sum_{j \in N} (p_{2,j,1} + p_{2,j,2} + p_{2,j,3}) + I_{y^*}\}$

The jobs in $N_1 \cup N_2$ are sequenced in Step 4 of Algorithm JOB3 in such a way as to minimize the idle time created by the jobs in N_3 .

Some comments about the recurrence relation now follow. The next job, j , is chosen according to the order in which Johnson's algorithm selects jobs for scheduling. From the definition of S , if $j \in S \cap A$, then A_{12} is increased by $p_{j,1}$, and A_{22} is increased by $p_{j,2}$. Also, the time from the completion of A_{12} to the completion of A_{22} is increased to $A_{22} - A_{12} + p_{j,2}$ if that quantity is larger than D_{A2} (see Figure 8). Identical changes occur for $j \in W \cap A$. From the definition of T , if $j \in T \cap A$, then A_{11} is increased by $p_{j,1}$, and A_{21} is increased by $p_{j,2}$. Also, the time from the completion of A_{11} to the completion of A_{21} is increased to the larger of $D_{A1} + p_{j,2} - p_{j,1}$ and $p_{j,2}$ (see Figure 8). Identical changes occur for $j \in V \cap A$. The changes that occur for $j \in B$ are symmetric.

Let $p_{\max} = \max_{j \in N'; i=1,2,3} \{p_{j,i}\}$. The main result of this section now follows.

Theorem 6

Algorithm JOB3 finds an optimal schedule for problem $J2|n_j \leq 3|C_t$ under Condition (1) in $O(n^{13}(p_{\max})^{11})$ time.

Proof

Algorithm JOB3 compares the cost of all possible state transitions. Only those partial schedules that are dominated are eliminated in Step 2. Consequently, JOB3 finds an optimal schedule.

The number of possible values for A_{11} is $np_{\max}$. The same is true for the other six state variables that are shown explicitly in $F(\cdot, \cdot, \cdot, \cdot, \cdot, \cdot, \cdot, \cdot)$. Thus, the number of possible states is $O((np_{\max})^7)$. The value of D_{A1} is no larger than n times the processing time of the largest operation, which is $np_{\max}$. The same is true for D_{A2}, D_{B1} and D_{B2} . Consequently, for each state, there are $O((np_{\max})^4)$ partial schedules. Hence, the number of schedules examined is $O((np_{\max})^{11})$. The modifications to each schedule in Step 1 can be implemented in constant time. Also, in Step 2, each possible schedule is eliminated at most once. Since the number of (d, j) pairs examined in Steps 1 through 3 is $O(n^2)$, the time complexity required for these steps is $O(n^{13}(p_{\max})^{11})$. Step 4 requires $O(n)$ time for each undominated schedule. Finally, Steps 5 and 6 require only constant time for each undominated schedule. It follows that the overall time complexity of JOB3 is $O(n^{13}(p_{\max})^{11})$. $\square$

From Theorem 6, we conclude that the recognition version of problem $J2|n_j \leq 3|C_t$ under Condition (1) is not unary NP-complete, although it is binary NP-complete (Theorem 5).

We observe that Condition (1) is stronger than is necessary to develop a pseudopolynomial time algorithm. The exact requirement needed is that there exists a sequence of the jobs of N' that is consistent with the order of selection by Johnson's rule, whether the job is assigned to set A or B . Consequently, it is sufficient that the data satisfies the following five, collectively weaker but less concise, conditions:

- 1'. $p_{j,1} \leq p_{s,1} \Leftrightarrow p_{j,3} \leq p_{s,3}$ for $j, s \in T$.
- 2'. $p_{j,1} \leq p_{s,1} \Leftrightarrow p_{j,2} \leq p_{s,2}$ for $j, s \in V$.
- 3'. $p_{j,2} \leq p_{s,2} \Leftrightarrow p_{j,3} \leq p_{s,3}$ for $j, s \in W$.
- 4'. $p_{k,1} \geq p_{j,1}$ and $p_{k,3} \leq p_{s,3} \Rightarrow p_{j,2} \leq p_{s,2}$ for $k \in T, j \in V, s \in W$.
- 5'. $p_{k,1} \leq p_{j,1}$ and $p_{k,3} \geq p_{s,3} \Rightarrow p_{j,2} \geq p_{s,2}$ for $k \in T, j \in V, s \in W$.

The first three conditions ensure that there is a consistent job order within T , within V , and within W . Set S automatically has a consistent job order because the second operation is always the smallest. Once Conditions 2' and 3' are satisfied, $V \cup W$ has a consistent job order. Conditions 4' and 5' ensure that $T \cup (V \cup W)$ has a consistent job order.

To obtain the order of selection of the jobs, we index the jobs such that $p_{j,2} \leq p_{j+1,2}$ for all $j \in N' \setminus T$. Then, each $k \in T$ is merged into this list between the elements of $V \cup W$ using Conditions 4' and 5'. Observe that the jobs of S are scheduled in blocks A_{12}, A_{22}, B_{21} , and B_{31} , while the jobs of T are scheduled in blocks A_{11}, A_{21}, B_{22} , and B_{33} . As a result, we arbitrarily insert the jobs of T among the jobs of S .

6. OPEN SHOPS

From Proposition 1 and the subsequent discussion, it is sufficient to consider stable schedules. Lee and Posner [4] show that the cycle time of flow shops is equal to the maximum machine workload, regardless of the processing order. Therefore, sequencing the jobs in arbitrary index order is optimal. As a result, an optimal sequence for the flow shop problem can be found in constant time for any number of machines. Also, an optimal schedule including the start or completion times of all operations can be found in $O(\sum_{j \in N} n_j)$ time.

In open shops, the order in which the jobs visit the machines is not restricted. Consequently, the operation sequence for each job and each machine must be determined. The

following algorithm finds an optimal schedule for cyclic open shop problems with any number of machines.

Algorithm OPEN

1. Order each job on each machine in index order. If the job is not processed on the machine, then go to the next machine.
2. Process each job as soon as possible.
3. Output the final schedule.

Theorem 7

For cyclic open shops with any number of machines, Algorithm OPEN finds an optimal schedule in $O(\sum_{j \in N} n_j)$ time. Further, the optimal cycle time of the schedule is equal to the maximum workload.

Proof

In any cyclic shop, the cycle time is no less than the maximum workload of any machine. For a cyclic open shop, since Algorithm OPEN sequences the jobs in the same order on all machines, the schedule is equivalent to a flow shop schedule. It follows from Lee and Posner [4] that the cycle time is the maximum workload of any machine, regardless of the operation sequences. Thus, the schedule is optimal.

The overall time complexity of Algorithm OPEN is given by the time required to find the start or completion times of all the $O(\sum_{j \in N} n_j)$ operations. $\square$

7. FINAL REMARKS

It is interesting to compare the solvability of scheduling problems with makespan and cycle time objectives. Theorem 1 shows that an optimal sequencing rule for problem $J|n_j \leq 2|C_t$ runs in polynomial time. The same is true for the optimal sequencing rule for problem $J2|n_j \leq 2|C_{\max}$ [22]. However, problems $J2|n_j \leq 3|C_{\max}$ and $J3|n_j \leq 2|C_{\max}$ are binary NP-hard [19, 23], as is problem $J2|n_j \leq 3|C_t$. Problem $J3|n_j \leq 3|C_t$ is unary NP-hard (see Theorem 4), as is problem $J3|n_j \leq 3|C_{\max}$ [24].

Although we have characterized cyclic open shops and most classes of job shops, it remains open whether problem $J2|n_j \leq k|C_t$, where $k = 3$ or 4 , is solvable in pseudopolynomial time. Research is also needed on cyclic job shop models with limited buffer capacity or limited wait restrictions. It would also be interesting, following the work of Matsuo [9], to consider the problem of minimizing the makespan of each job set while still maintaining the maximum throughput rate.

ACKNOWLEDGEMENTS

The research of the first author is supported in part by National Science Foundation under grant number DMI-9821033 and in part by the Summer Fellowship Program of the Max M. Fisher College of Business, The Ohio State University. The research of the third author is supported by the National Science Foundation under grant number DMI-9908437. Two anonymous referees provided helpful comments on earlier drafts of this paper.

REFERENCES

1. Hitz KL. Scheduling of flexible flow shops–II, *Technical Report LIDS-R-1049*, Laboratory for Information and Decision Systems, MIT, Cambridge, MA, 1980.
2. Gershwin SB, Akella R, Choong YF. Short-term production scheduling of an automated manufacturing facility. *IBM Journal of Research and Development* 1985; **29**:392–400.
3. Chaar JK, Davidson ES. Cyclic job shop scheduling using reservation tables. *Proceedings of the 1990 IEEE International Conference on Robotics and Automation*, Cincinnati, OH, 1990; 2128–2135.
4. Lee T-E, Posner ME. Performance measures and schedule patterns in periodic job shops. *Operations Research* 1997; **45**:72–91.
5. Hanen C. Study of an NP-hard cyclic scheduling problem: the periodic recurrent job shop. *European Journal of Operational Research* 1994; **72**:82–101.
6. Garey MR, Johnson DS. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman: San Francisco, 1979.
7. Carlier J, Chrétienne P. On the dominance of K-periodic schedules in cyclic scheduling. *Second International Workshop on Project Management and Scheduling*, Compiègne, France, 1990.
8. McCormick ST, Pinedo ML, Shenker S, Wolf B. Sequencing in an assembly line with blocking to minimize cycle time. *Operations Research* 1989; **37**:925–935.
9. Matsuo H. Cyclic sequencing problems in the two-machine permutation flowshop: complexity, worst case and average case analysis. *Naval Research Logistics* 1990; **37**:679–694.
10. Hall NG, Sriskandarajah C. A survey of machine scheduling problems with blocking and no-wait in process. *Operations Research* 1996; **44**:510–525.
11. Graves SC, Meal HC, Stefek D, Zeghmi AH. Scheduling of re-entrant flow shops. *Journal of Operations Management* 1983; **3**:197–207.
12. Lev V, Adiri I. V-shop scheduling. *European Journal of Operational Research* 1984; **18**:51–56.
13. Claver JF, Jackson P. Lot sizing in cyclic scheduling. *Technical Report No. 736*, School of Operations Research and Industrial Engineering, Cornell University, Ithaca, NY, 1987.
14. Roundy R. Cyclic schedules for job-shops with identical jobs. *Mathematics of Operations Research* 1992; **17**:842–865.
15. Wang MY, Sethi SP, van de Velde SL. Minimizing makespan in a class of reentrant shops. *Operations Research* 1997; **45**:702–712.
16. Kamoun H, Sriskandarajah C. The complexity of scheduling jobs in repetitive manufacturing systems. *European Journal of Operational Research* 1993; **70**:350–364.
17. McCormick ST, Rao US. Some complexity results in cyclic scheduling. *Mathematical and Computer Modelling* 1994; **20**:107–122.
18. Zhang H, Graves SC. Cyclic scheduling in a stochastic environment. *Operations Research* 1997; **45**:894–903.
19. Lenstra JK, Rinnooy Kan AHG, Brucker P. Complexity of machine scheduling problems. *Annals of Discrete Mathematics* 1977; **1**:343–362.
20. Graham RL, Lawler EL, Lenstra JK, Rinnooy Kan AHG. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics* 1979; **5**:287–326.
21. Johnson SM. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly* 1954; **1**:61–68.
22. Jackson JR. An extension of Johnson's results on job lot scheduling. *Naval Research Logistics Quarterly* 1956; **3**:201–203.
23. Gonzalez T, Sahni S. Flowshop and jobshop schedules: complexity and approximation. *Operations Research* 1978; **26**:36–52.
24. Garey MR, Johnson DS, Sethi R. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research* 1976; **1**:117–129.